

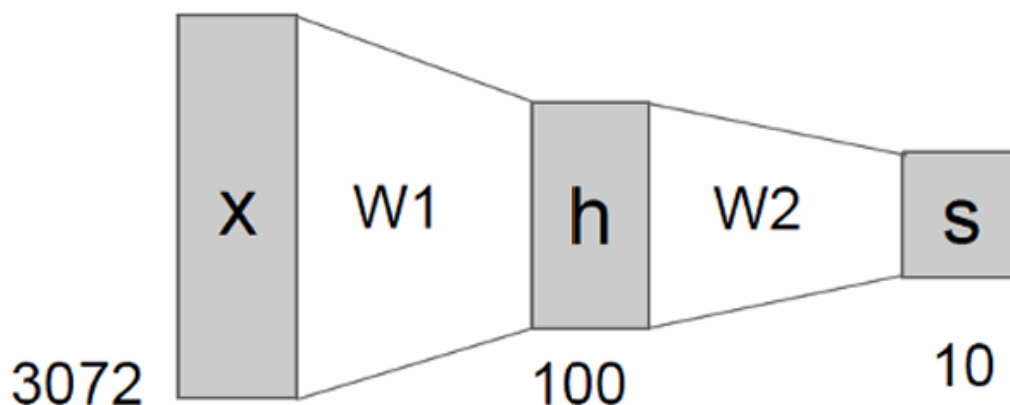
- [1. 神经网络 \(Neural network\)](#)
 - [1.1 激活函数 \(Activation Functions\)](#)
 - [1.2 神经网络的架构](#)
 - [1.3 神经网络的过程](#)
 - [1.3.1 链式法则 \(Chain Rule\)](#)
 - [1.3.1.1 三种基本节点的反向传播模式](#)
 - [1.3.1.2 池化的反向传播](#)
 - [1.3.1.3 向量时的梯度计算](#)

1. 神经网络 (Neural network)

这一部分知识在另一个功课里详细讨论了，这里讨论的神经网络是数学模型，而不是生物学上的大脑。

传统的线性模型： $f = Wx$

这样的模型难以学习复杂的函数映射，因此有神经网络： $f = W_2 \max(0, W_1 x)$ (两层)

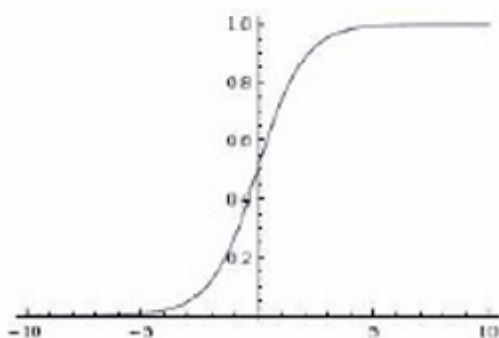


或者三层神经网络： $f = W_3 \max(0, W_2 \max(0, W_1 x))$ 。

1.1 激活函数 (Activation Functions)

神经网络里有多种激活函数 (Activation Functions)，我们快速带过：

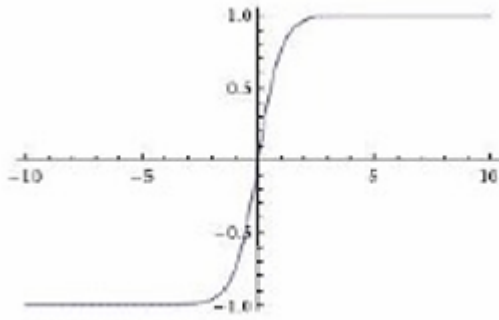
1. Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$



S形曲线，输出值在0到1之间。

平滑，输出值范围有限，但容易受到梯度消失问题的影响。

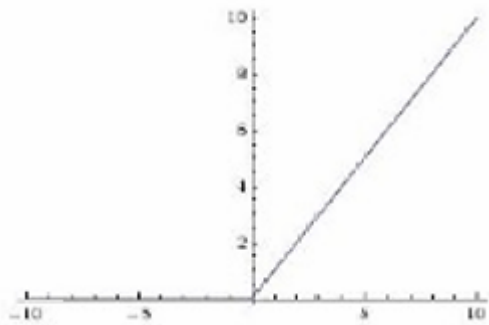
2. tanh: $\tanh(x)$



S形曲线，输出值在-1到1之间。

相对于Sigmoid，tanh的输出中心对称，但同样可能遇到梯度消失问题。

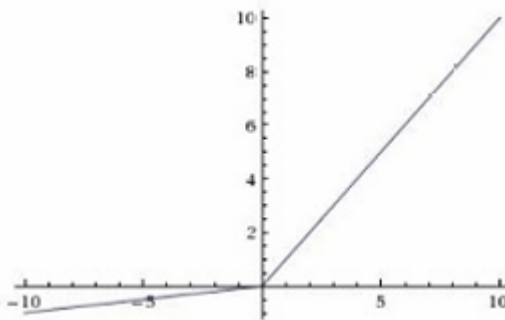
3. ReLU (Rectified Linear Unit): $\max(0, x)$



在 $x=0$ 处有一个转折点， $x>0$ 时输出为 x ， $x\leq 0$ 时输出为0。

简单，计算效率高，有助于缓解梯度消失问题，但可能导致“死亡ReLU”问题（即神经元永久失活）。

4. Leaky ReLU: $\max(0.1x, x)$



在 $x<0$ 时有一个小的斜率（0.1）， $x\geq 0$ 时斜率为1。

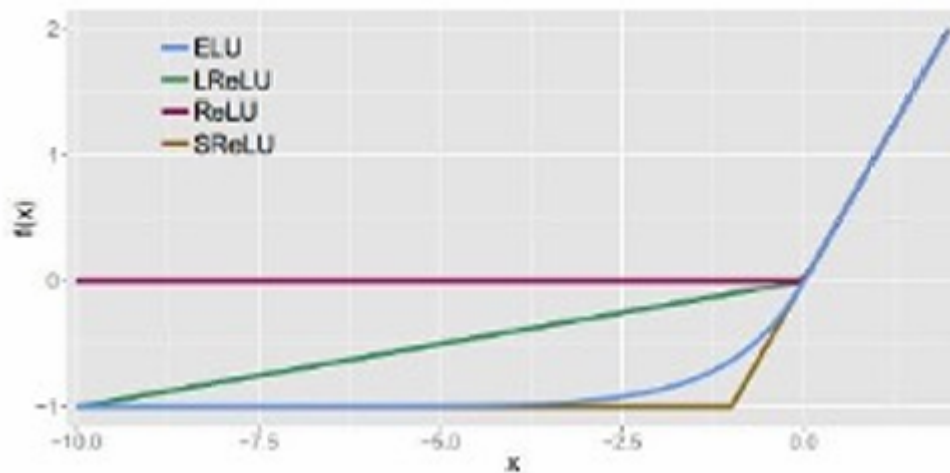
改进了ReLU，允许负值有一个小的梯度，有助于解决“死亡ReLU”问题。

5. Maxout: $\max(w_1^T x + b_1, w_2^T x + b_2)$

是一种通用的激活函数，可以看作是ReLU和Leaky ReLU的推广，通过选择两个线性函数的最大值来激活。

6. ELU (Exponential Linear Unit):

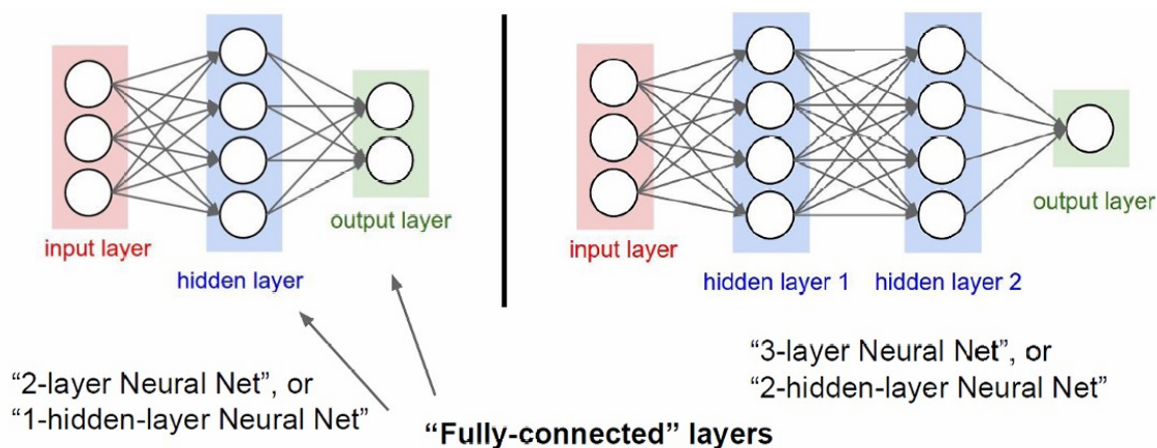
$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$



在 $x > 0$ 时输出为 x ，在 $x \leq 0$ 时输出为 $\alpha(\exp(x) - 1)$ 。

类似于Leaky ReLU，但使用指数函数来处理负值，有助于解决梯度消失问题，并且可以提供更好的性能。

1.2 神经网络的架构



主要由三个部分组成：

输入层 (input layer)：接收输入数据。

隐藏层 (hidden layer)：处理输入数据，并通过激活函数引入非线性。

输出层 (output layer)：产生最终的输出结果。

这里左边是单一隐藏层，右边是两层隐藏层。

每一层的每个神经元都与下一层的所有神经元相连，这种连接方式是全连接的。

1.3 神经网络的过程

前向传播是指数据从输入层经过隐藏层最终到达输出层的过程，其中每一层的输出成为下一层的输入。这里给出代码示例。

```
f = lambda x: 1.0/(1.0 + np.exp(-x))
x = np.random.randn(3, 1)
h1 = f(np.dot(w1, x) + b1)
h2 = f(np.dot(w2, h1) + b2)
out = np.dot(w3, h2) + b3
```

这里定义了一个匿名函数 f ，使用Sigmoid函数作为激活函数。

然后生成一个包含3个随机数的列向量，作为输入层的输入。

再使用使用权重矩阵 $W1$ 和偏置向量 $b1$ 计算第一隐藏层的激活值。这里 $\text{np.dot}(W1, x)$ 计算输入和权重的点积，然后加上偏置，最后通过激活函数 f 处理。

接着使用权重矩阵 W_2 和偏置向量 b_2 计算第二隐藏层的激活值，其中 h_1 是第一隐藏层的输出。
最后使用权重矩阵 W_3 和偏置向量 b_3 计算输出层的激活值，其中 h_2 是第二隐藏层的输出。这里没有应用激活函数，假设输出层直接输出线性组合的结果。

前向传播的下一步是损失计算。

线性模型的输出： $s = f(x; W) = Wx$

支持向量机 (SVM) 的损失函数： $L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$ 这个公式表示对于每个样本，计算其分数与其他所有样本分数的差异，如果这个差异大于1，则计算损失。

总损失函数： $L = \frac{1}{N} \sum_{i=1}^N L_i + \sum_k W_k^2$ ，由两部分组成：

第一部分是数据损失，即所有样本损失的平均值。

第二部分是正则化项，用于防止过拟合。这里使用的是L2正则化，即权重的平方和。

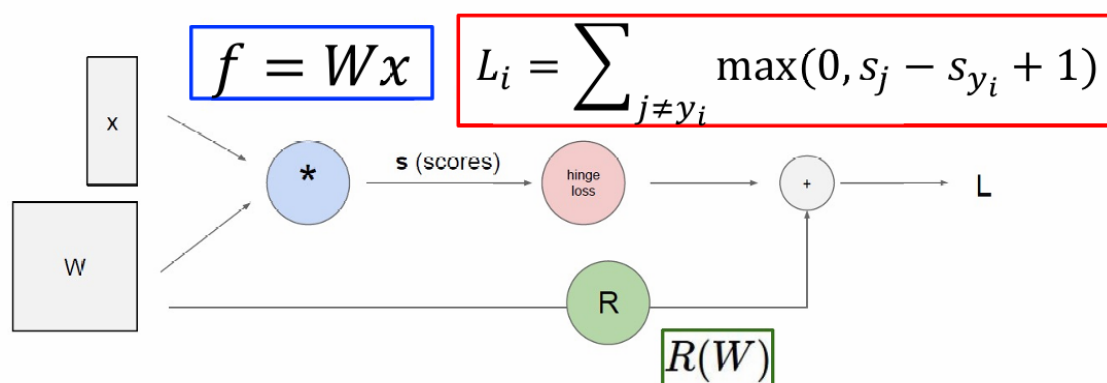
计算完损失函数进入下一步后向传播 (Backpropagation) ，

这里使用梯度下降来优化损失函数，这么做的目标是计算这个梯度 $\nabla_W L$ ，以便更新权重，从而最小化损失函数。

利用链式法则从输出层向输入层反向计算梯度。

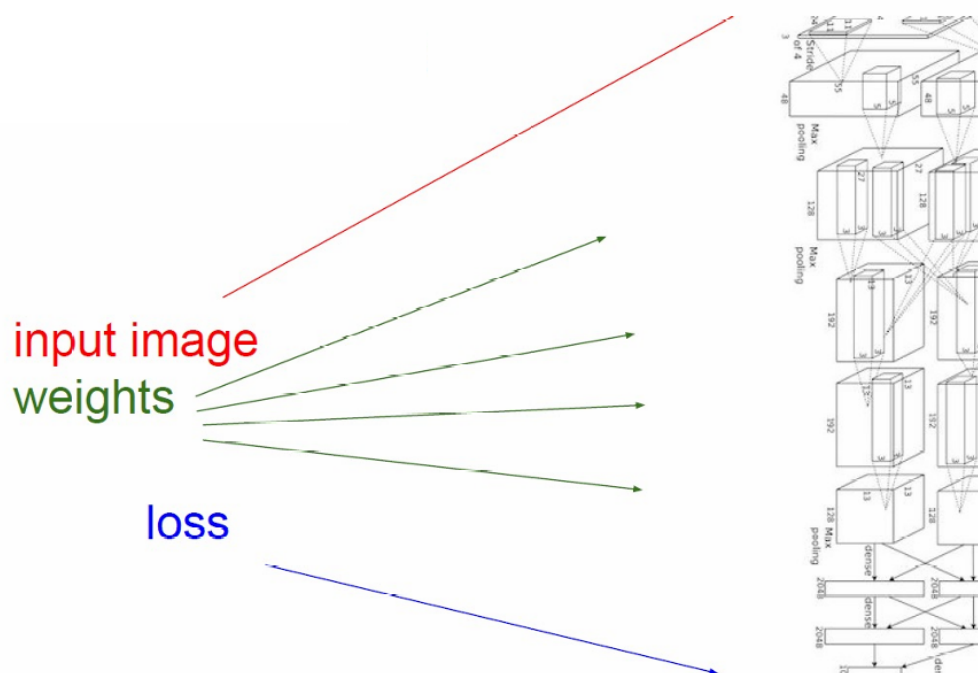
这一步的目的是确定如何调整网络参数以减少损失。

因此前向传播的计算过程如下图所示。

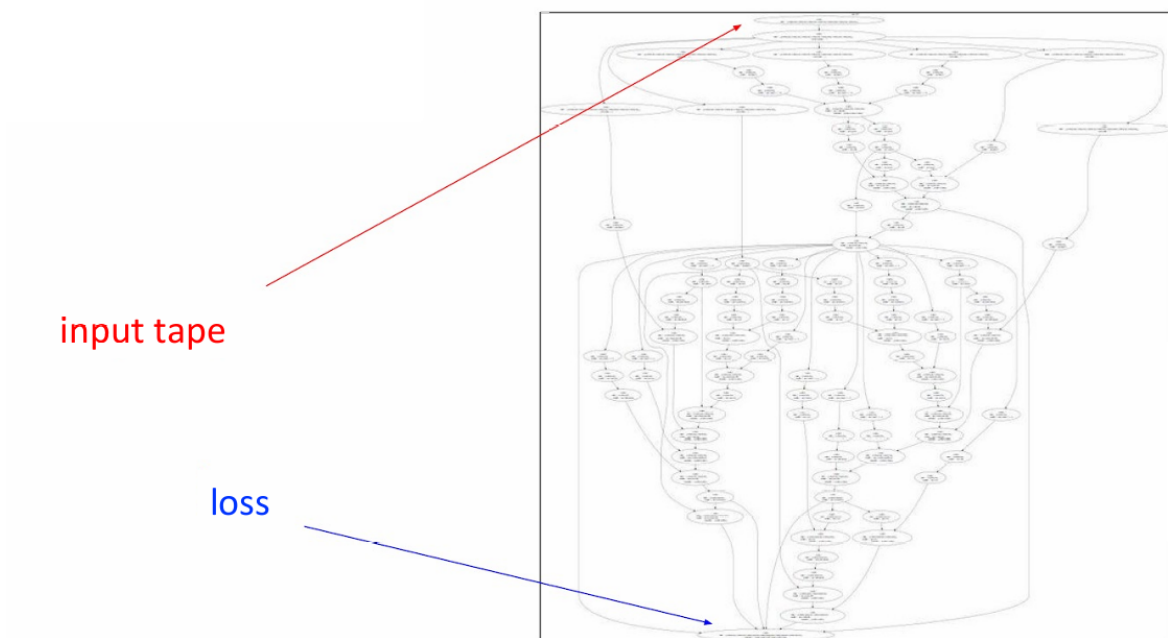


先通过输入和权重计算分数，然后算出单个样本的SVM损失，然后和正则化项一起得到总损失。

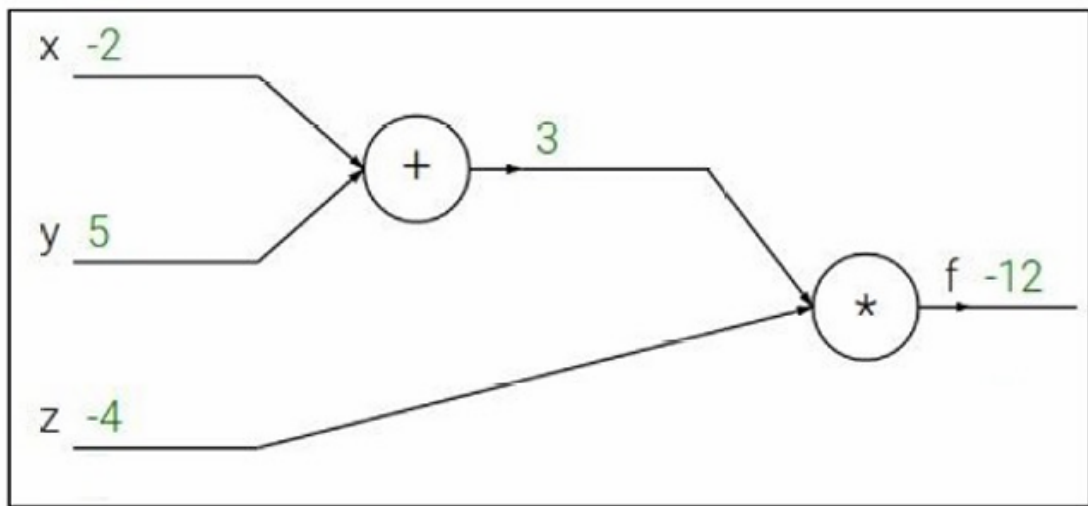
卷积神经网络 (AlexNet架构) 的过程如下图所示。



神经图灵机 (Neural Turing Machine, NTM) 的过程如下图所示。



我们来看一个例子。



定义函数: $f(x, y, z) = (x + y)z$

$$q = x + y$$

$$f = qz$$

输入: $x = -2$

$$y = 5$$

$$z = -4$$

所以计算可以得到: $q = 3$

$$f = -12$$

计算完后完成前向传播, 我们现在计算暂时跳过计算损失函数, 我们这个例子默认里面存在损失, 现在我们利用反向传播更新网络参数以减少损失。

所以我们需要计算损失函数相对于输入变量 x , y , 和 z 的梯度。在梯度下降优化算法中, 需要计算损失函数相对于模型参数 (包括输入变量) 的梯度, 以确定如何调整这些参数来减少损失。

对于 $q = x + y$,

$$\frac{\partial q}{\partial x} = 1$$

$$\frac{\partial q}{\partial y} = 1$$

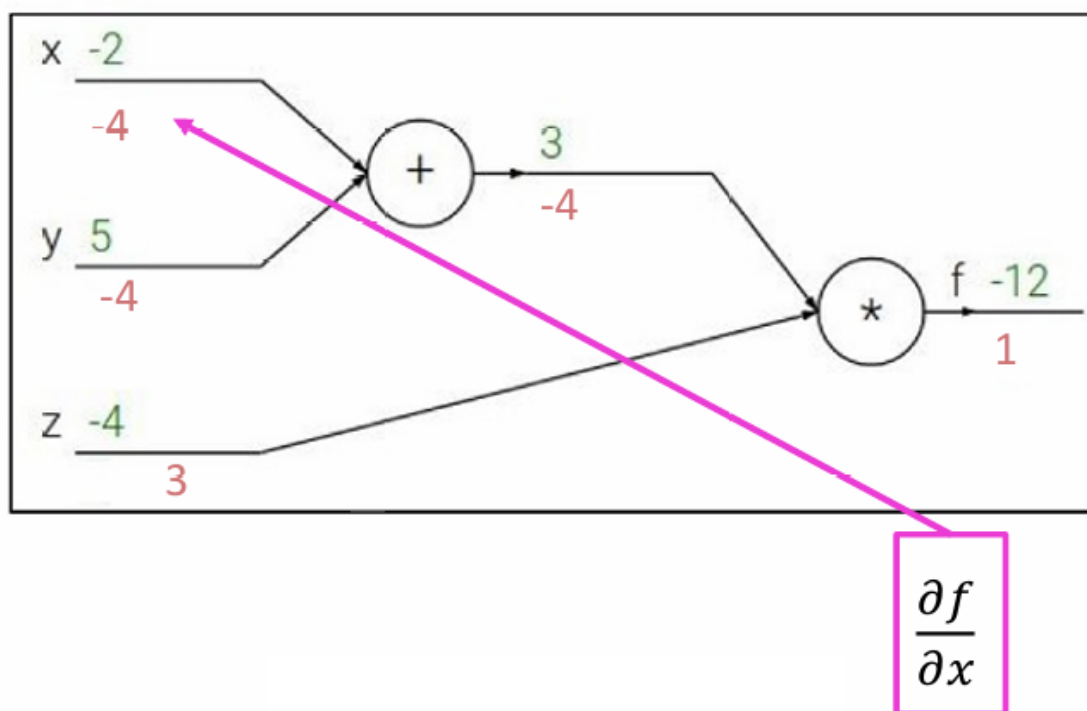
对于 $f = qz$,

$$\frac{\partial f}{\partial q} = z$$

$$\frac{\partial f}{\partial z} = q$$

计算 $\frac{\partial f}{\partial y}$ 使用链式法则，可以得到 $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y} = z$
 计算 $\frac{\partial f}{\partial x}$ 使用链式法则，可以得到 $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = z$

因此最后的结果如下图所示。



然后我们可以根据这里计算得到的梯度来更新网络中的权重。

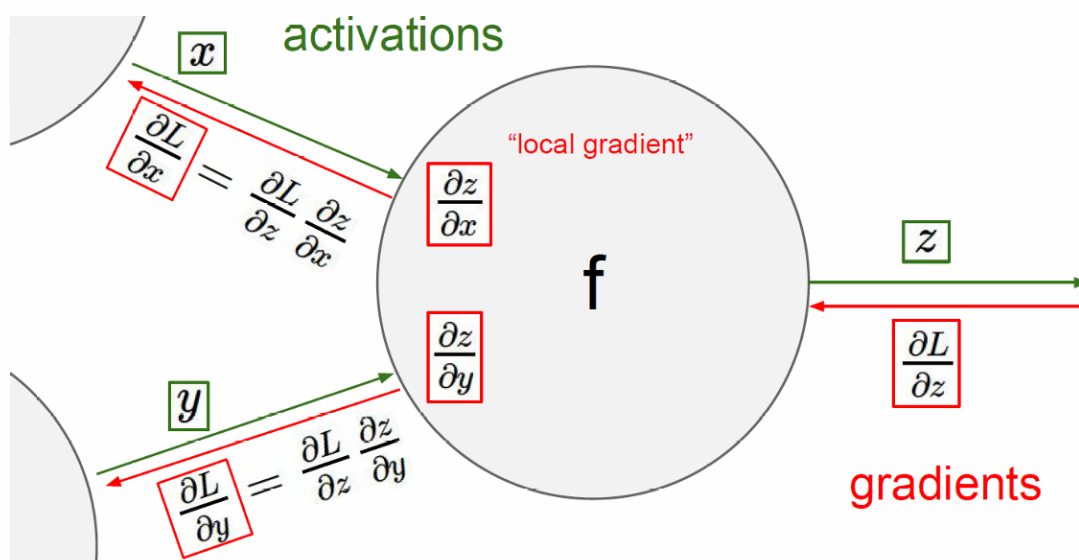
1.3.1 链式法则 (Chain Rule)

链式法则是微积分中用于计算复合函数导数的一个基本法则，我们回顾一下。

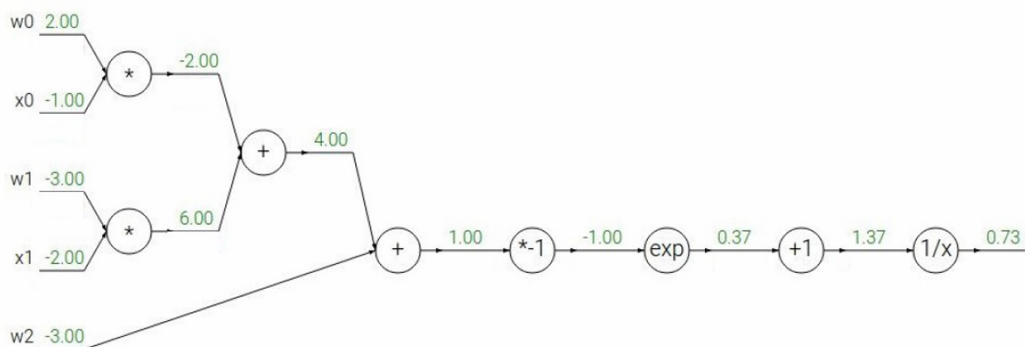
使用链式法则可以通过将损失函数相对于中间变量 z 的梯度与中间变量 z 相对于输入 x 和 y 的梯度相乘，来计算损失函数相对于输入 x 和 y 的梯度。

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial y}$$



下面我们尝试一个比较复杂的例子。



函数: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$

我们先接受输入计算 $z = w_0x_0 + w_1x_1 + w_2$

$$z = 2.00 \times (-1.00) + (-3.00) \times (-2.00) + (-3.00) = -2.00 + 6.00 - 3.00 = 1.00$$

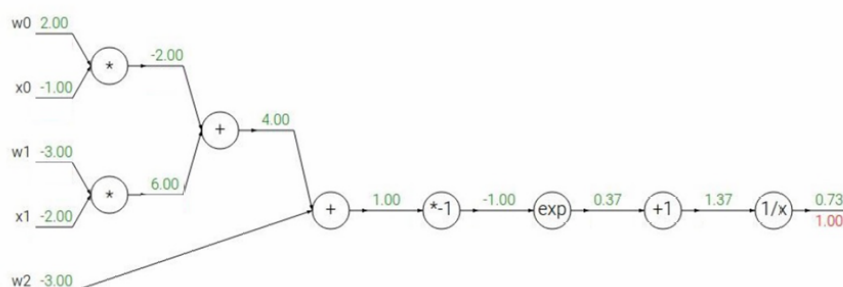
因此 $-z = -1$

$$\exp(-z) = \exp(-1.00) \approx 0.37$$

$$\text{计算 } \exp(-z) + 1: \exp(-1.00) + 1 \approx 0.37 + 1 = 1.37$$

$$\text{最后, 计算 } \frac{1}{x}: \frac{1}{1.37} \approx 0.73$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



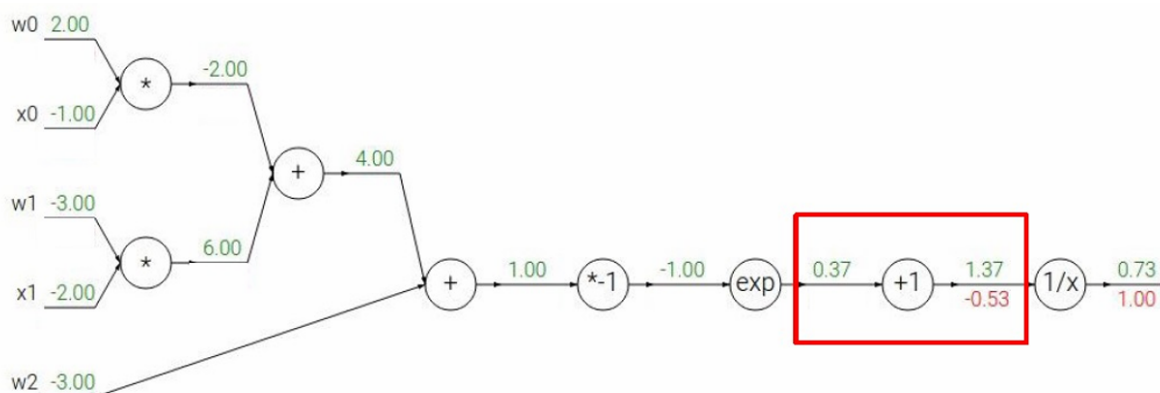
先看一下基础函数的导数: $f(x) = e^x \Rightarrow \frac{df}{dx} = e^x$

$$f_a(x) = ax \Rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \Rightarrow \frac{df}{dx} = -\frac{1}{x^2}$$

$$f_c(x) = c + x \Rightarrow \frac{df}{dx} = 1$$

然后我们开始反向传播, 首先是起点, f 对 f 的梯度, 即: $\frac{\partial f}{\partial f} = 1.00$

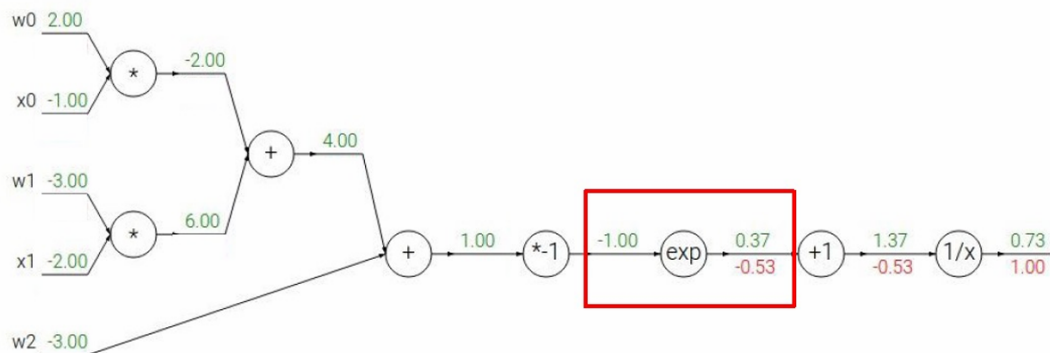


然后下一步, 经过 $f = \frac{1}{x}$ 节点, 我们要计算 $\frac{\partial L}{\partial a}$

$$\text{根据链式法则 } \frac{\partial L}{\partial a} = \frac{\partial f}{\partial a} \frac{\partial L}{\partial f}$$

$$\text{因为 } \frac{\partial f}{\partial a} = -\frac{1}{a^2}$$

$$\text{所以 } \frac{\partial L}{\partial a} = -\frac{1}{a^2} \frac{\partial L}{\partial f} = -\frac{1}{1.37^2} (1.00) = -0.53$$

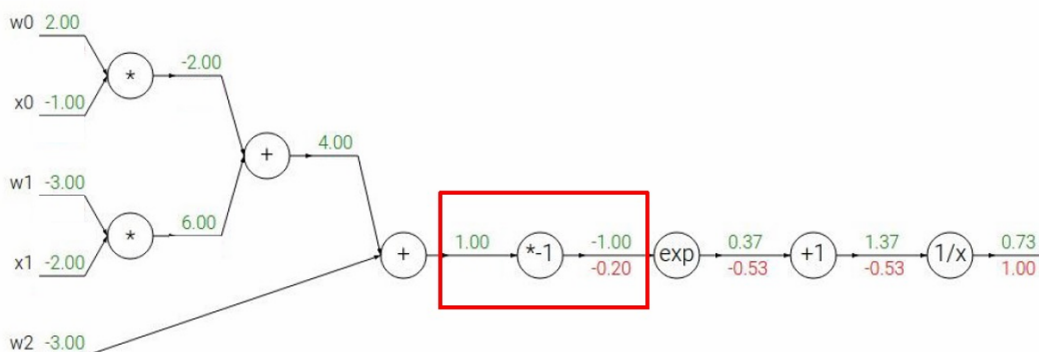


然后下一步，经过 $a = 1 + t$ 节点，我们要计算 $\frac{\partial L}{\partial t}$

根据链式法则 $\frac{\partial L}{\partial a} = \frac{\partial a}{\partial t} \frac{\partial L}{\partial a}$

因为 $\frac{\partial a}{\partial t} = 1$ (因为 $a = 1 + t$ 对 t 的导数为 1)

所以 $\frac{\partial L}{\partial t} = 1 \times (-0.53) = -0.53$

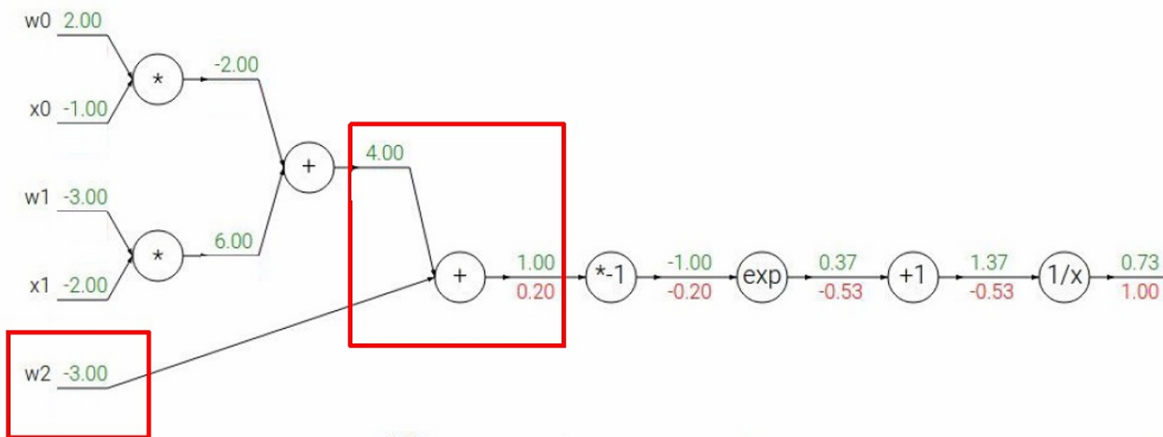


然后下一步，经过 $t = e^u$ 节点，我们要计算 $\frac{\partial L}{\partial u}$

根据链式法则 $\frac{\partial L}{\partial u} = \frac{\partial t}{\partial u} \frac{\partial L}{\partial t}$

因为 $\frac{\partial t}{\partial u} = e^u$

所以 $\frac{\partial L}{\partial u} = e^u \times \frac{\partial L}{\partial t} \approx 0.37 \times (-0.53) \approx -0.196$

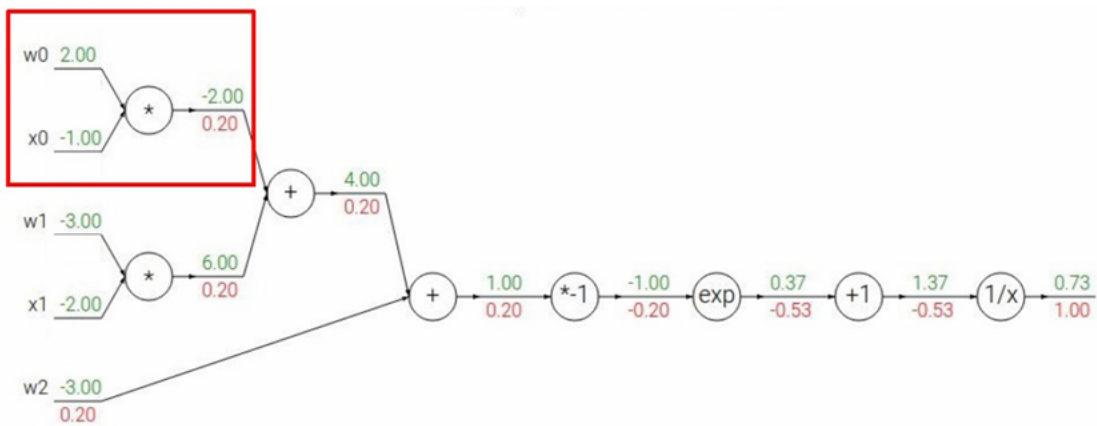


然后下一步，经过 $u = -z$ 节点，我们要计算 $\frac{\partial L}{\partial z}$

根据链式法则 $\frac{\partial L}{\partial z} = \frac{\partial u}{\partial z} \frac{\partial L}{\partial u}$

因为 $u = -z$

所以 $\frac{\partial L}{\partial z} = (-1) \times (-0.20) = 0.20$



然后下一步，经过 $z = A + B + C$ 节点，其中： $A = w_0x_0$

$$B = w_1x_1$$

$$C = w_2$$

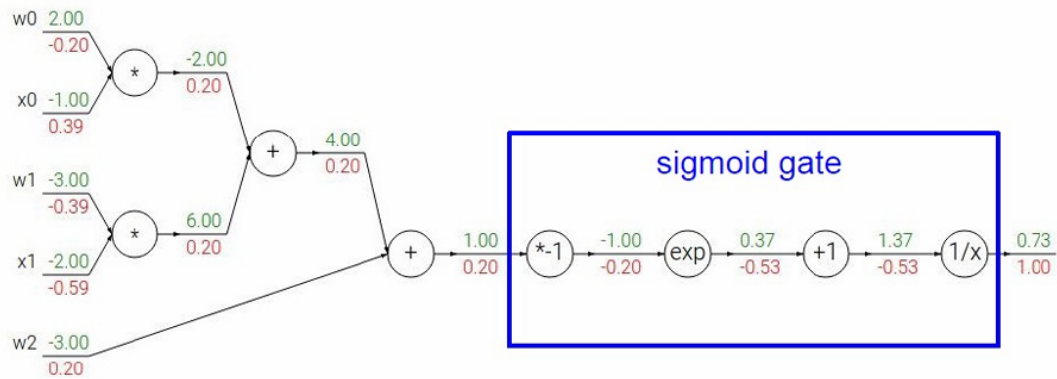
我们要计算 $\frac{\partial L}{\partial A}$, $\frac{\partial L}{\partial B}$, $\frac{\partial L}{\partial C}$

根据链式法则 $\frac{\partial L}{\partial A} = \frac{\partial z}{\partial A} \frac{\partial L}{\partial z} = 1 \times 0.20 = 0.20$

$$\frac{\partial L}{\partial B} = \frac{\partial z}{\partial B} \frac{\partial L}{\partial z} = 1 \times 0.20 = 0.20$$

$$\frac{\partial L}{\partial C} = \frac{\partial z}{\partial C} \frac{\partial L}{\partial z} = 1 \times 0.20 = 0.20$$

也就是局部梯度 (local gradient) 乘以上游梯度 (its gradient, 从后面传回来的梯度)



然后下一步，经过 $w_0 \times x_0$ 节点，其中： $A = w_0x_0$

$$w_0 = 2.00$$

$$x_0 = -1.00$$

我们要计算 $\frac{\partial L}{\partial w_0}$, $\frac{\partial L}{\partial x_0}$

根据链式法则 $\frac{\partial L}{\partial w_0} = \frac{\partial A}{\partial w_0} \frac{\partial L}{\partial A} = x_0 \times 0.20 = (-1.00) \times 0.20 = -0.20$

$$\frac{\partial L}{\partial x_0} = \frac{\partial A}{\partial x_0} \frac{\partial L}{\partial A} = w_0 \times 0.20 = 2.00 \times 0.20 = 0.40$$

类似地，我们可以计算得到另一边 $w_1 \times x_1$ 节点的结果是

$$\frac{\partial L}{\partial w_1} = \frac{\partial B}{\partial w_1} \frac{\partial L}{\partial B} = x_1 \times 0.20 = (-2.00) \times 0.20 = -0.40$$

$$\frac{\partial L}{\partial x_1} = \frac{\partial B}{\partial x_1} \frac{\partial L}{\partial B} = w_1 \times 0.20 = -3.00 \times 0.20 = 0.60$$

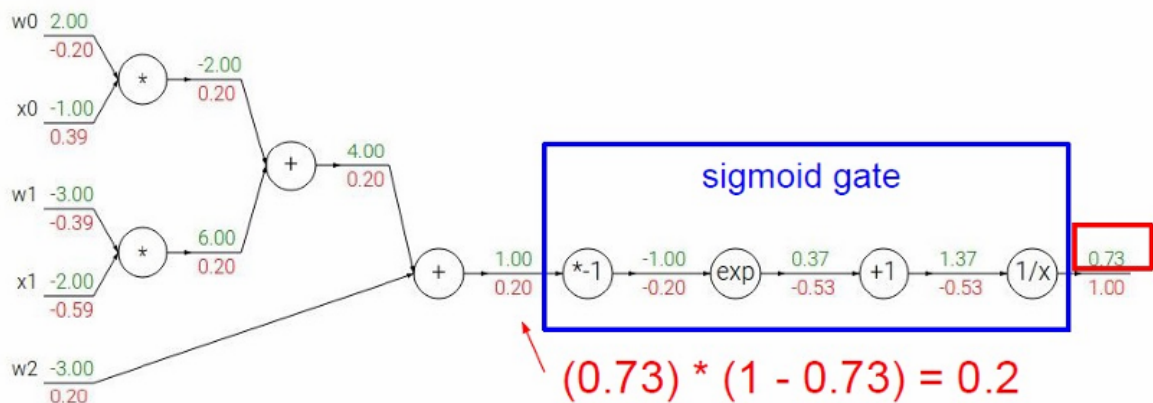
当然我们这里是一步一步通过链式法则解决的，我们也可以将其看作一个完整的函数也就是将整个 Sigmoid 函数看作一个计算节点 (gate)。

$$\text{Sigmoid 函数: } \sigma(x) = \frac{1}{1+e^{-x}}$$

$$\text{Sigmoid 的导数为: } \frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

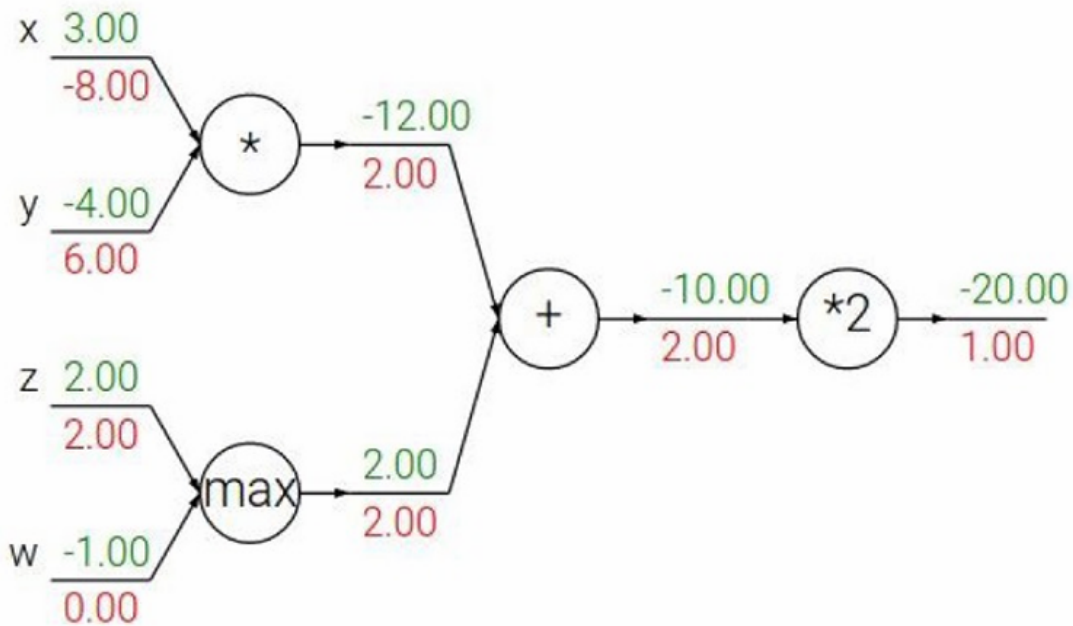
$$\text{推导过程: } \sigma'(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1+e^{-x}-1}{1+e^{-x}} = \sigma(x)(1 - \sigma(x))$$

$$\text{计算结果: } (0.73) * (1 - 0.73) = 0.2$$



这与我们之前反向传播得到的结果一致。

1.3.1.1 三种基本节点的反向传播模式



有三种基本节点：

1. 加法节点 (add gate)：梯度分配器 (gradient distributor)

特点：将上游梯度原样分配给所有输入

原理： $z = x + y$, $\frac{\partial z}{\partial x} = 1$, $\frac{\partial z}{\partial y} = 1$

例子：上游梯度为 2.00 ，两个输入都得到 2.00

2. 乘法节点 (mul gate)：梯度交换器 (gradient switcher)

特点：将上游梯度乘以另一个输入的值后传递

原理： $z = x \times y$, $\frac{\partial z}{\partial x} = y$, $\frac{\partial z}{\partial y} = x$

例子： $\frac{\partial L}{\partial x} = y \times 2.00 = -4.00 \times 2.00 = -8.00$

$\frac{\partial L}{\partial y} = x \times 2.00 = 3.00 \times 2.00 = 6.00$

3. 最大值节点 (max gate)：梯度路由器 (gradient router)

特点：只将梯度传递给较大的输入，较小输入得到零梯度

原理： $z = \max(x, y)$

如果 $x > y$ ，则 $\frac{\partial z}{\partial x} = 1$, $\frac{\partial z}{\partial y} = 0$

如果 $y > x$ ，则 $\frac{\partial z}{\partial x} = 0$, $\frac{\partial z}{\partial y} = 1$

例子中，因为 x 较大所以 $\frac{\partial L}{\partial x} = 2.00$, $\frac{\partial L}{\partial y} = 0.00$

1.3.1.2 池化的反向传播

1. Lp池化 (Lp-pooling)

$$y = (\sum_i x_i^p)^{\frac{1}{p}}, \text{ 其中 } x_i > 0.$$

$$\text{已知: } y' = \frac{\partial L}{\partial y}$$

$$\text{求: } x'_i = \frac{\partial L}{\partial x_i}$$

$$\text{我们设 } S = \sum_i x_i^p, \text{ 则 } y = S^{\frac{1}{p}}$$

$$\text{先求 } \frac{\partial y}{\partial x_i}:$$

$$\frac{\partial y}{\partial x_i} = \frac{\partial}{\partial x_i} S^{\frac{1}{p}} = \frac{1}{p} S^{\frac{1}{p} - 1} \cdot p x_i^{p-1} = S^{\frac{1}{p} - 1} x_i^{p-1}$$

$$\text{由于 } y = S^{\frac{1}{p}}, \text{ 所以 } S^{\frac{1}{p} - 1} = \frac{y}{S} = \frac{y}{\sum_j x_j^p}.$$

$$\text{因此: } \frac{\partial y}{\partial x_i} = \frac{y}{\sum_j x_j^p} x_i^{p-1}$$

$$\text{最终: } x'_i = \frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x_i} = y' \cdot \frac{y}{\sum_j x_j^p} x_i^{p-1}$$

2. 对数平均池化 (log-average module)

$$y = \frac{1}{\beta} \ln \left(\frac{1}{n} \sum_i \exp(\beta x_i) \right)$$

$$\text{已知: } y' = \frac{\partial L}{\partial y}$$

$$\text{求: } x'_i = \frac{\partial L}{\partial x_i}$$

$$\text{我们设 } S = \frac{1}{n} \sum_i \exp(\beta x_i), \text{ 则 } y = \frac{1}{\beta} \ln S.$$

$$\text{先求 } \frac{\partial y}{\partial x_i}:$$

$$\frac{\partial y}{\partial x_i} = \frac{1}{\beta} \cdot \frac{1}{S} \cdot \frac{\partial S}{\partial x_i} = \frac{1}{\beta} \cdot \frac{1}{S} \cdot \frac{1}{n} \cdot \beta \exp(\beta x_i)$$

$$\frac{\partial y}{\partial x_i} = \frac{\exp(\beta x_i)}{nS}$$

$$\text{由于 } S = \frac{1}{n} \sum_j \exp(\beta x_j), \text{ 且 } \beta y = \ln S, \text{ 所以 } S = \exp(\beta y).$$

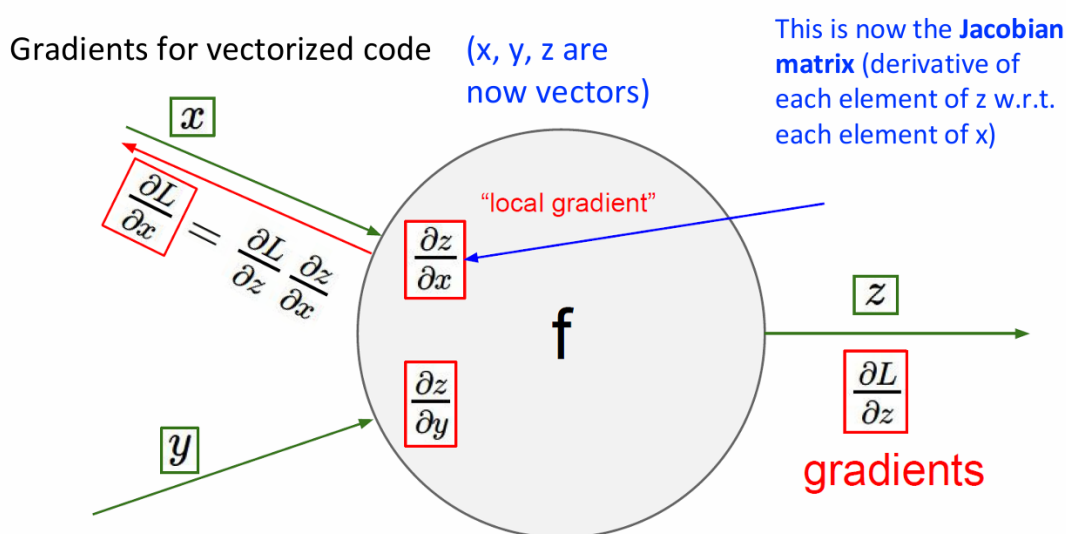
$$\text{因此: } \frac{\partial y}{\partial x_i} = \frac{\exp(\beta x_i)}{n \exp(\beta y)} = \frac{\exp(\beta(x_i - y))}{n}$$

$$\text{最终: } x'_i = \frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x_i} = y' \cdot \frac{\exp(\beta(x_i - y))}{n}$$

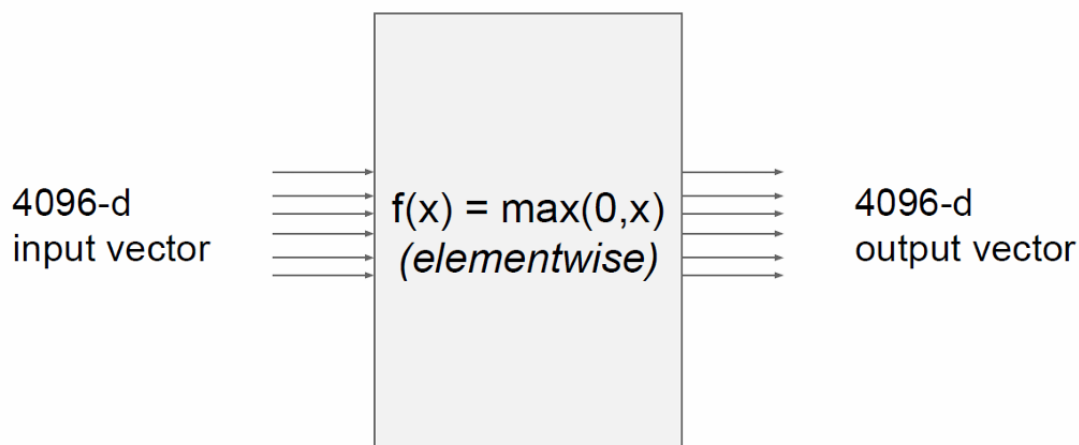
1.3.1.3 向量时的梯度计算

在之前的例子中, x, y, z 都是标量 (单个数值)。现在考虑它们都是向量的情况。

我们用雅可比矩阵来描述向量值函数变化率, 其中每个元素是 z 的一个分量对 x 的一个分量的偏导数。



向量化操作如下。



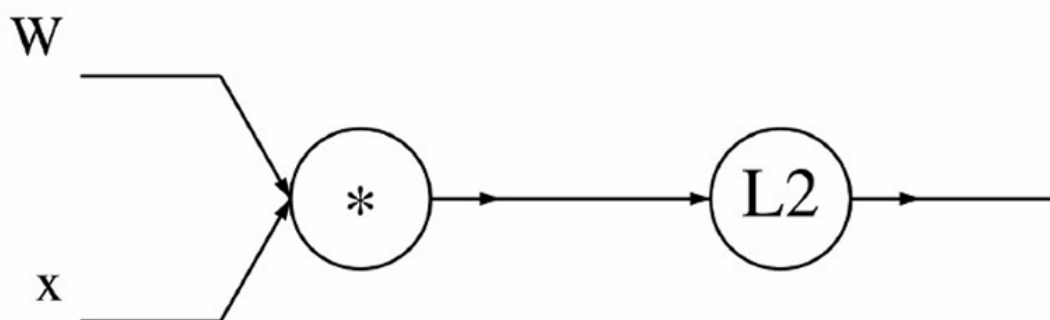
由于输入和输出都是4096维向量，且每个输出元素都是输入元素的逐元素操作的结果，因此雅可比矩阵是一个4096x4096的矩阵。这是因为每个输出元素对每个输入元素都有一个偏导数，形成了一个完整的矩阵。

我们这里使用的函数是ReLU函数，此时雅可比矩阵是一个稀疏矩阵，其中大部分元素为0，只有当输入元素 $x_i > 0$ 时，对应的偏导数为1。因此，雅可比矩阵中只有少数元素为1，其余为0。

理论上，如果考虑整个minibatch，雅可比矩阵的大小将是 409600x409600。这是因为每个输入向量有4096个元素，minibatch中有100个这样的向量，所以总的输入维度是 $4096 \times 100 = 409600$ ，输出也是如此。

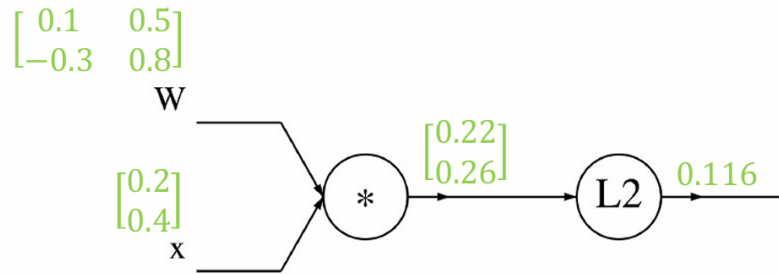
在实际应用中，尽管理论上雅可比矩阵的大小是 409600x409600，但在实践中，由于ReLU函数的特性（其导数在正值区域为1，在非正值区域为0），雅可比矩阵通常是稀疏的。这意味着在计算梯度时，我们只需要关注那些输入值为正的元素，从而大大减少了计算量，提升了计算效率。

我们现在看一个例子，用向量化操作计算L2范数（也称为欧几里得范数）的平方



$f(x, W) = \|W \cdot x\|^2 = \sum_{i=1}^n (W \cdot x)_i^2$ ，其中 x 是一个 R^n 维的向量， W 是一个 $R^{n \times n}$ 维的矩阵。我们给定输入值矩阵，如下图所示。

A vectorized example: $f(x, W) = \|W \cdot x\|^2 = \sum_{i=1}^n (W \cdot x)_i^2$

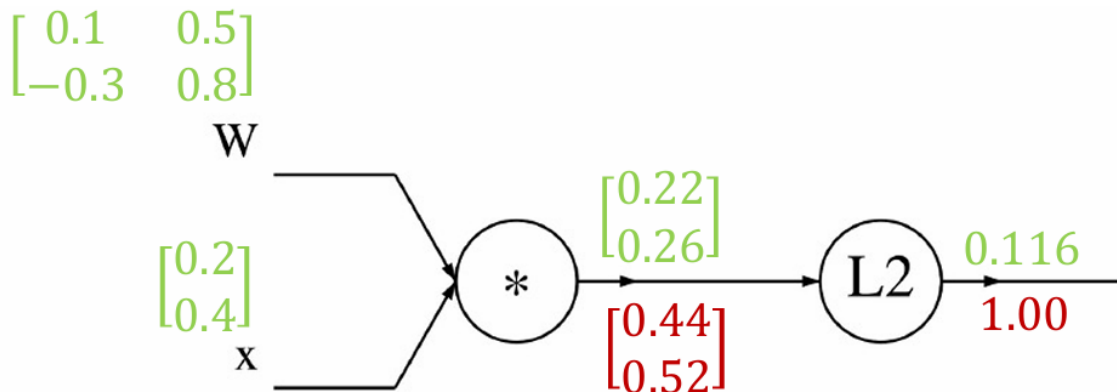


$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \dots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \dots + W_{n,n}x_n \end{pmatrix}$$

$$f(q) = \|q\|^2 = q_1^2 + \dots + q_n^2$$

我们先进行前向计算:

$$q = Wx = \begin{bmatrix} 0.1 \times 0.2 + 0.5 \times 0.4 \\ -0.3 \times 0.2 + 0.8 \times 0.4 \end{bmatrix} = \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix}, \quad f = \|q\|^2 = 0.22^2 + 0.26^2 = 0.116$$

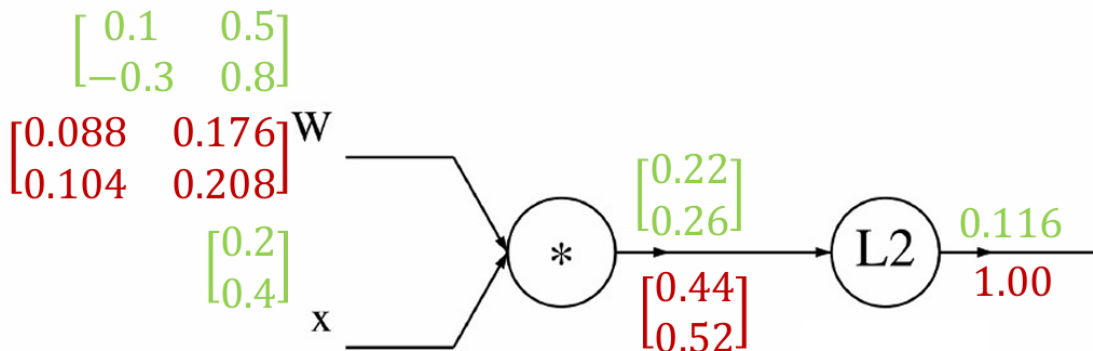


再计算反向传播梯度。

从损失 f 开始, $\frac{\partial f}{\partial f} = 1.00$

计算梯度: $\frac{\partial f}{\partial q_i} = \frac{(q_1^2 + q_2^2)}{\partial q_i} = 2q_i$

$$\nabla_q f = 2q = \begin{bmatrix} 0.44 \\ 0.52 \end{bmatrix}$$



我们现在计算 f 对权重矩阵 W 的梯度。

使用链式法则: $\frac{\partial f}{\partial W_{i,j}} = \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial W_{i,j}}$

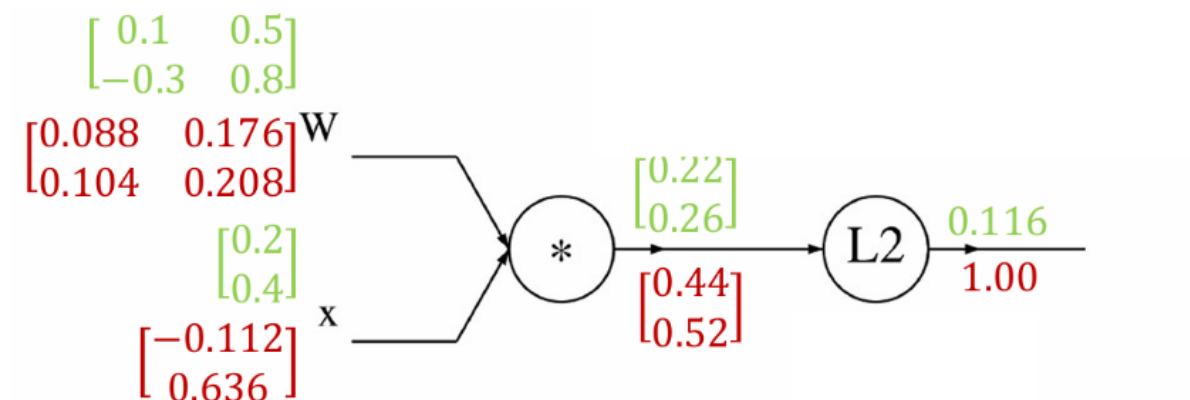
其中: $\frac{\partial f}{\partial q_k} = 2q_k$

$\frac{\partial q_k}{\partial W_{i,j}} = 1_{k=i} x_j (q_k = \sum_l W_{k,l} x_l, \frac{\partial q_k}{\partial W_{i,j}} = \begin{cases} x_j & \text{if } k = i \\ 0 & \text{otherwise} \end{cases})$

代入得: $\frac{\partial f}{\partial W_{i,j}} = \sum_k (2q_k) (1_{k=i} x_j) = 2q_i x_j$

矩阵形式: $\nabla_W f = 2q x^\top$

所以 $\nabla_W f = 2 \times \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} \begin{bmatrix} 0.2 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.088 & 0.176 \\ 0.104 & 0.208 \end{bmatrix}$



我们现在计算输入向量 x 对权重矩阵 W 的梯度。

使用链式法则: $\frac{\partial f}{\partial x_i} = \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial x_i}$

其中: $\frac{\partial f}{\partial q_k} = 2q_k$

$\frac{\partial q_k}{\partial x_i} = W_{k,i} (q_k = \sum_l W_{k,l} x_l)$

代入得: $\frac{\partial f}{\partial x_i} = \sum_k (2q_k) W_{k,i}$

矩阵形式: $\nabla_x f = 2W^\top q$

所以 $\nabla_x f = 2 \times \begin{bmatrix} 0.1 & -0.3 \\ 0.5 & 0.8 \end{bmatrix} \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} = \begin{bmatrix} -0.112 \\ 0.636 \end{bmatrix}$